

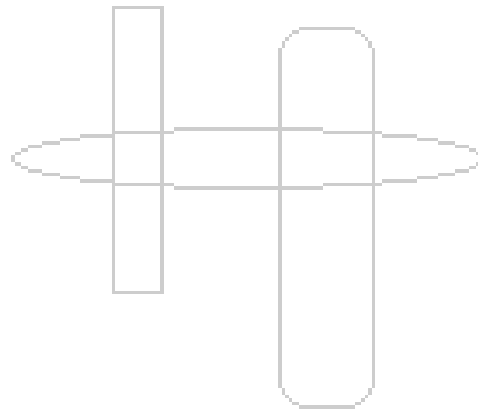
Title: Course Structure

FF No. 653

Branch: Computer **Year:** S.Y. **A.Y.:** 2025-26 **Module:** IV

Sub No.	Subject Code	Subject Name	Teaching Scheme (Hrs/Week)			Examination Scheme										Total	Credits
			Th	Lab	Tut	Test -1	MSA	Test -2		ESA							
						CT1 (%)	MSE (%)	Test (%)	HA (%)	LAB CA (%)	CP (%)	PPT /GD (%)	Practical + CVV (%)	CVV (%)	ESE (%)		
S1	CS2305	PCC: Data Structures-II	2	2	0					10	30		40+20			100	3
S2	CS2306	PCC: Theory of Computation	2	0	0	35		35						30			2
S3	CS2307	PCC: Operating System	2	2	0					10	30		40+20			100	3
S4	CS2308	PCC: Software Engineering	2	2	0					10	30			20	40	100	3
S5	CS2309	PCC: Mobile App Development	0	2	0					10	20				60	100	1
S6	CSM***	MDM: Microcontroller	2	-	-	35 (O)		35 (O)	30							100	3
S7	HSS2003	HSS:															2
S8	HS2002	AEC: Reasoning And Aptitude Development - 4													100 (O)	100	1
S9	CS2231	VSEC: Design Thinking - 4													100	100	1

S10	CS2210	FP: Engineering Design And Innovation - IV					30								70	100	2
		Total															21



CS2305: Data Structures-II

Credits: 3

Teaching Scheme Theory: 2 Hours/Week

Lab: 2 Hours/Week

Course Prerequisites: Data Structures, Basic programming Skills (C/C++).

Course Objectives:

1. To Learn how and when to use data structures like AVL trees, red-black trees, B-trees, Fibonacci heaps, and others.
2. To Apply advanced data structures to solve complex computational problems efficiently.
3. To Gain hands-on experience in implementing and using these structures through programming assignments.
4. To Foster the ability to design new data structures or improve existing ones for specialized needs.
5. To prepare students for algorithm-heavy roles that require knowledge of advanced structures.

Course Relevance:

The course on Advanced Data Structures is highly relevant for several reasons, particularly in the context of computer science, software engineering, and related fields. The course is essential for anyone aiming to build efficient, scalable, and optimized software systems or preparing for roles in software development, data science, and academia. Advanced data structures (e.g., segment trees, tries, B-trees, red-black trees, Fibonacci heaps) are essential for optimizing time and space complexity in complex algorithms. Understanding advanced data structures is crucial for solving high-level problems in coding competitions and technical interviews. Used in databases (e.g., B-trees in indexing), compilers (e.g., syntax trees), operating systems (e.g., scheduling queues), and networks (e.g., tries in routing). Key for designing scalable systems (e.g., caching systems, real-time data processing) where performance and memory management are critical. Advanced structures are often the basis for innovations in algorithms, machine learning, data mining, and bioinformatics.

Syllabus

Theory

Section 1: Advanced Trees, Priority Queues and DS for Strings.

Unit 1: Advanced Trees and Applications

(8 Hours)

Threaded Binary Tree, AVL Tree, Red-Black Tree, Heap Tree, Huffman Tree. B-Tree, B+-Tree, Splay Tree, Van Emde Boas Tree, Fusion Tree, Dynamic Finger Search Trees with Time and Space Complexity Analysis

Unit 2: Priority Queues and Heaps

(4 Hours)

Double Ended Priority queues, Leftist Trees, Binomial Heaps, Fibonacci Heaps, skew heaps, pairing heaps

Unit 3: Data Structures for Strings

(4 Hours)

String Searching: preliminaries, the DAWG, the position Heaps, tries and compressed tries, Suffix Trees and suffix arrays, Dictionaries Allowing Errors in Queries.

Section 2: Randomized, Multidimensional and Miscellaneous Data Structures

Unit 4: Randomized Data Structures

(6 Hours)

Preliminaries of randomized algorithm and probability theory, Skip Lists: Structural Properties of Skip Lists, Space Complexity of skip list. Treap: A Randomized Binary Search Tree

Unit 5: Multidimensional Spatial Data Structures

(5 Hours)

Introduction, point data, region data and Rectangle data. Quad trees and Octrees: Quad trees for point data, spatial queries with region quad tree. Interval trees, Segment trees, Range trees, and Priority Search Trees. Binary Space Partitioning Trees, R-trees.

Unit 6: Miscellaneous Data Structures

(3 Hours)

Google's Big Table, Data Structures for Sets: The Disjoint Set Union-Find Problem, Concurrent Data structures, Succinct Representation of Data Structures: Bit vector, Succinct Dictionaries, Tree Representations. Persistent data structures, Cache-Oblivious Data Structure.

**Syllabus
Laboratory**

List of Experiments

1. Assignment based on TBT creation and performing Stackless Traversals.
2. Assignment based on operations on AVL and RED-Black trees
3. Assignment based on B Trees and B+ Trees.
4. Assignment based on Priority Queues Application
5. Assignment based on tries.
6. Assignment based on Suffix Trees.
7. Assignment Based on Randomized Data Structures.
8. Assignment based on Quad trees and Oct trees
9. Assignment based on Interval trees, Segment trees, Range trees.
10. Assignment based on R-trees.
11. Assignment Based on Disjoint Set data structures.

12. Assignment based on concurrent data structures.
13. Assignment based on Succinct data structures.

Syllabus Course Project

List of Course Projects

- 1) Performance Comparison of AVL and Red-Black Trees on Dynamic Data Sets
- 2) Visualizing Rotations in AVL and Red-Black Trees
- 3) Designing a Priority Queue System Using Red-Black Trees
- 4) Implementing a Self-Balancing Dictionary Using AVL Trees
- 5) Implementation of Binomial Heaps and Their Use in Priority Queues
- 6) Comparative Study: Binomial Heap vs Binary Heap vs Fibonacci Heap
- 7) Task Scheduling Simulation Using Binomial Heaps
- 8) Analysis of Union and Decrease-Key Operations in Binomial Heaps
- 9) Implementing a Simple Database Index Using B-Trees
- 10) File System Directory Management Using B+ Trees
- 11) Comparison of Search Performance: B-Tree vs B+ Tree on Disk-Based Data
- 12) Building a Block-Oriented Key-Value Store Using B+ Trees
- 13) Implementation and Performance Analysis of R-Trees for Spatial Indexing
- 14) Design and Application of KD-Trees for Nearest Neighbor Search
- 15) Spatial Query Optimization Using Quadtrees in 2D Geographic Data
- 16) Building a 3D Game World Using Octrees for Scene Management
- 17) Multidimensional Range Search Using Range Trees
- 18) Efficient Spatial Joins Using Grid-Based Indexing
- 19) Comparison of Spatial Indexing Techniques: R-Tree vs. QuadTree vs. KD-Tree
- 20) Real-Time Object Tracking Using Spatial Hashing
- 21) Clustering Geospatial Data Using k-d Trees and Voronoi Diagrams
- 22) Designing a Mini GIS Engine with Support for Spatial Queries

Course Outcomes

The student will be able to –

- 1) Analyze and implement advanced binary tree structures.
- 2) Model the real world problem with the help of appropriate priority queues and heap data structure.
- 3) Construct and evaluate and string processing structures for real world applications.
- 4) Apply randomized data structures for optimized performance.
- 5) Model and query spatial and multidimensional data effectively.
- 6) Critically evaluate the specialized and modern data structures for complex real-world problems.

CO-PO Mapping

	Program Outcomes (PO)											PSO		
CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	3								2	3		2
CO2	2	3	2								2	3		2
CO3	3	3	3			2	2				2	3	2	2
CO4	3	3	3	3		2	2				2	3	2	2
CO5	3	3	2								2	3		2
CO6	2	3	3								2	3		2
Average	2.5	3.0	2.66	3.0		2.0	2.0				2.0	3.0	2.0	2.0

CO Attainment levels:

Weights for attainment levels: L1 - Easy-0.75 L2 - Comfortable-0.7 L3 – Medium – 0.65

L4 – Somewhat difficult – 0.6 L5 – Difficult – 0.55

CO1 – L3, CO2– L3, CO3 – L2, CO4 – L4, CO5 – L4 and CO6 – L5

Future Courses Mapping:

Design and Analysis of Algorithms, Computational Geometry, Data Mining and Information Retrieval, Machine Learning and Data Science, Parallel and Distributed Computing, Compiler Design, Advanced Topics in Data Structures and Algorithms.

Job Mapping:

Students who have completed a course in Advanced Data Structures has a bright career paths and roles where the knowledge and skills from the course are directly applicable are Software Developer / Software Engineer, Data Engineer, Backend Developer, Database Developer / Administrator, Algorithm Engineer, Competitive Programmer , Coding Coach, Computer Science Researcher, System Programmer, Operating System Developer, Machine Learning Engineer / AI Engineer, Big Data Engineer / Cloud Engineer, Cyber security Analyst / Cryptography Engineer, Game Developer / Graphics Programmer.

This job mapping shows that Advanced Data Structures is a core competency across multiple high-demand domains in both industry and academia.

Text Books:

1. Sartaj Sahni, Dinesh P. Mehta; *Handbook of Data Structures and Applications*;

2nd edition, Chapman & Hall/CRC.

2. *Fundamentals of Data Structures in C*, E. Horwitz, S. Sahani, Anderson-Freed, Second Edition, Universities Press.

Reference Books:

1. T. Cormen, R. Rivest, C. Stein, C. Leiserson, “*Introduction to Algorithms*”, Second Edition, PHI publication.

2. Peter Brass, *Advanced Data Structures*, First Edition, Cambridge University Press.

For MOOCs and other learning Resources

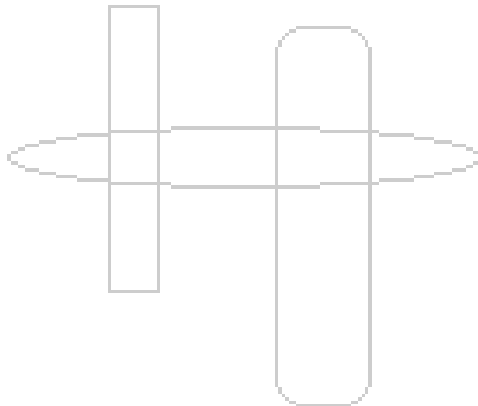
1. www.nptelvideos.in,

2. www.geeksforgeeks.org

3. <https://www.youtube.com/watch?v=244YpoG1pqA&list=PLrikLQMZHuSonRoDheibeb9ffd9phWIyu&index=5>

4. <https://classroom.volp.in/>

5. <https://www.coursera.org/>



CS2306: Theory of Computation

Credits: 2

Teaching Scheme Theory: 2 Hours/Week

Course Prerequisites: Introduction to discrete mathematics, proof techniques, basic familiarity with programming/computing.

Course Objectives:

1. Students will learn basic concepts such as alphabet, strings, Languages, Decision problems, etc and will be able to work with the abstract formal setup.
2. Students will be able to design deterministic/nondeterministic automata for regular languages, also he will be able to prove non regularity of languages through application of Pumping Lemma and Myhill-Nerode theorem.
3. Students will gain understanding of the role of non-determinism in Automata theory.
4. Students will be able to design Context free grammars, Push down automata for context Free Languages
5. Students will be able to design Turing Machines for various computational problems and see the equivalence of TM models with high level programming languages.
6. Students will be able to comprehend the meaning of undecidability in the context of Turing Machine Model and understand the inherent limits of computation.

Course Relevance:

This is a foundational course for Computer Science and Engineering. The central theme of the course is to study what makes certain computational problems very hard and the others easy? Is there some concrete theoretical evidence for the exhibited hardness of the problems? The course explores these questions, first by introducing students to the abstract notion of computation and models of computation. Starting from very simple model of state machines to finally cumulating into the Turing machine model (which is a foundation of modern-day computers), several models in between are studied. For every model, questions such as, which computational problems can be/cannot be solved in the model? How efficiently a problem can be solved in a particular model? various closure properties of the model are studied. Throughout the course emphasis is given to proving things with concrete mathematical arguments.

The course is very important for understanding the concept of computation in a more abstract set-up. Wherever one wants to formally talk about the underlying model, the restrictions imposed by the model, what is the power and limitation of the model, the principles learnt in this course are useful. Due to the abstract nature of the course, the principles learnt have wide applicability. The course is an essential prerequisite for several advanced courses such as Computational Complexity, Advanced Algorithms, Foundation of Logic, Quantum Computation, Parallel computation, Circuit Complexity etc. On a more applied side: The Automata theoretic models, concept of Context Free Grammar and

Pushdown Automata studied in the course are very important for Compiler design. The models discussed during the course have direct applications to several machine learning models, Natural Language processing, Artificial Intelligence, Functional Programming.

Once the student gains expertise in thinking abstractly about underlying models of computation it facilitates systematic study of any other domain (in computer science or otherwise) which demands logical thinking and abstraction.

This course is also relevant for students who want to pursue a research career in theory of computing, computational complexity theory, Natural Language Processing, advanced algorithmic research.

Syllabus Theory

Section 1: Topics/Contents

Unit 1: Finite Automata

(4 Hours)

Introduction to Automata, Computability and Complexity theory, Automaton as a model of computation, Central Concepts of Automata Theory: Alphabets, Strings, Languages. Decision Problems Vs Languages. Finite Automata, Structural Representations, Deterministic Finite Automata (DFA)-Formal Definition, Simplified notation: State transition graph, transition table, Language of DFA, construction of DFAs for Languages and proving correctness, Product construction, Nondeterministic finite Automata (NFA), NFA with epsilon transition, Language of NFA, Conversion of NFA with epsilon transitions to DFA, Applications and Limitation of Finite Automata.

Unit 2: Regular and Non-Regular Languages

(6 Hours)

Regular expression (RE), Definition, Operators of regular expression and their precedence, Algebraic laws for Regular expressions, Kleene's Theorem: Equivalence Regular expressions and DFAs (without proof), Closure properties of Regular Languages (union, intersection, complementation, concatenation, Kleene closure), Decision properties of Regular Languages, Applications of Regular expressions. Myhill-Nerode theorem and applications: proving non-regularity, lower bound on number of states of DFA, State Minimization algorithm, Equivalence testing of DFAs. Non-Regular Languages, Revisiting Pigeon-Hole principle, Pumping Lemma for regular Languages.

Unit 3: Context Free Grammars

(4 Hours)

Context Free Grammars: Definition, Examples, Derivation, Languages of CFG, Constructing CFG, correctness proof using induction. Closure properties of CFLs (Union, Concatenation, Kleene closure, reversal). Derivation trees, Ambiguity in CFGs, Removing ambiguity, Inherent ambiguity. Normal forms for CFGs: CNF and GNF (without proof). Decision Properties of CFLs (Emptiness, Finiteness and Membership). Applications of CFG.

Section 2: Topics/Contents

Unit 4: Push Down Automata

(5 Hours)

Description and definition, Language of PDA, Acceptance by Final state, Acceptance by empty stack, Deterministic, Non-deterministic PDAs, CFG to PDA construction (with proof). Equivalence of PDA and CFG (without proof). Intersection of CFLs and Regular language. Pumping lemma for CFLs, non-Context Free Languages, Chomsky hierarchy.

Unit 5: Turing Machines

(5 Hours)

Basic model, definition, and representation, Instantaneous Description, Language acceptance by TM. Robustness of Turing Machine model and equivalence with various variants: Two-way/One-way infinite tape TM, multi-tape TM, non-deterministic TM, Universal Turing Machines. TM as enumerator. Recursive and Recursively Enumerable languages and their closure properties.

Unit 6: Introduction to Undecidability

(4 Hours)

Church-Turing Thesis and intuitive notion of Algorithm, Encoding for Turing machines and countability of set of all Turing machines. Existence of Turing unrecognizable languages via Cantor's diagonalization. Undecidability of Halting problem. Examples of undecidable problems: Post Correspondence Problem, Hilbert's 10th Problem, Tiling problem (without proof). Example of Turing unrecognizable language. Decision properties of R, RE languages.

Course Outcomes:

The student will be able to –

1. Infer the applicability of various automata theoretic models for recognizing formal languages.
2. Discriminate the expressive powers of various automata theoretic and formal language theoretic computational models.
3. Illustrate significance of non-determinism pertaining to expressive powers of various automata theoretic models.
4. Comprehend general purpose powers and computability issues related to state machines and grammars.
5. Explain the relevance of Church-Turing thesis, and the computational equivalence of Turing machine model with the general-purpose computers.
6. Grasp the theoretical limit of computation (independent of software or hardware used) via the concept of undecidability.

CO-PO Mapping

	Program Outcomes (PO)											PSO		
CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	3	3	1							2	3		
CO2	2	3	2								2	3		
CO3	2	3	2	2							2	3		
CO4	2	3									2	3		
CO5	2	3	2								2	3		
CO6	2	3		1							2	3		
Average	2	3	2.25	1.33							2	3		

Future Courses Mapping:

Compiler design, Computational Complexity theory, Computability theory, Advanced Algorithms, Natural Language Processing, Artificial Intelligence

Job Mapping:

The principles learnt in the course have wide applicability, in domains like Compiler design, Programming languages design, Machine learning, Natural Language processing, etc. Any job that involves modeling and systematic study of some systems, background of Theory of Computation is useful. Understanding of the course content is helpful in developing a systematic and structured approach towards programming. The programming/algorithm design abilities lie at the heart of computer science and are useful for several job profiles in the computer industry. If a student wants to pursue higher education/ research in Computer Science, this course is essential.

Books and E-Resources

Text Books: (As per IEEE format)

- 1.Hopcroft J, Motwani R, Ullman, Addison-Wesley, "Introduction to Automata Theory, Languages and Computation", Second Edition, ISBN 81-7808-347-7.
2. Michael Sipser, Course Technology, "Introduction to Theory of Computation", Third Edition, ISBN-10: 053494728X.
- 3.. "Discrete Mathematics and its applications" by Kenneth Rosen (William C Brown Publisher)

Reference Books: (As per IEEE format)

1. J. Martin, "Introduction to Languages and the Theory of Computation", Third edition, Tata McGraw-Hill, ISBN 0-07-049939-x, 2003.
2. Daniel I. A. Cohen, "Introduction to Computer Theory", Wiley-Second Edition, ISBN-10 : 04711377

For MOOCs and other learning Resources

www.nptelvideos.in

CS2307: Operating System

Credits: 3

Teaching Scheme Theory: 2 Hours/Week

Lab: 2 Hours/Week

Course Prerequisites:

1. Computer Architecture
2. Computer organization
3. Data Structure

Course Objectives:

1. To learn functions of Operating System
2. To learn the importance of concurrency and how to implement concurrent abstractions correctly in an OS.
3. To learn OS scheduling policies and mechanisms.
4. To deal with deadlock.
5. To learn memory management schemes in various ways to improve performance, and how this impacts system complexity.
6. To learn design & develop the Operating system from a scratch.

Course Relevance:

Operating Systems serve as the fundamental backbone of all computing systems, from mobile phones and laptops to enterprise servers and embedded systems. Understanding how operating systems function is essential for every computer engineer, as it enables them to efficiently manage hardware resources, develop robust software, and ensure system stability and performance.

Syllabus Theory

Section 1: Topics/Contents

Unit 1: Introduction to OS

(4 Hours)

What is OS, Interaction of OS and hardware, Goals of OS, Basic functions of OS, OS Services, System Calls, and Types of system calls. Types of OS: Batch, Multiprogramming, Time sharing, Parallel, Distributed & Real-time OS.

Unit 2: Process Management

(6 Hours)

Process Concept, Process States: 2, 5, 7 state models, Process Description, Process Control. **Threads:** Multithreading models, Thread implementations – user level and kernel level threads, Symmetric Multiprocessing. **Concurrency:** Issues with concurrency, Principles of **Concurrency Mutual**

Exclusion: H/W approaches, S/W approach, OS/Programming Language support: Semaphores, Mutex and Monitors. **Classical Problems of Synchronization:** Readers-Writers problem, Producer Consumer problem, Dining Philosopher problem

Unit 3: Process Scheduling

(4 Hours)

Uniprocessor Scheduling: Scheduling Criteria, Types of Scheduling: Pre-emptive, Non pre-emptive, Long-term, Medium-term, Short-term. **Scheduling Algorithms:** First Come First Serve, Shortest Job First, Round Robin, Priority.

Section 2: Topics/Contents

Unit 4: Deadlocks

(4 Hours)

Principles of deadlock, Deadlock Prevention, Deadlock Avoidance, Deadlock Detection, Deadlock Recovery

Unit 5: Memory Management

(6 Hours)

Fixed, Dynamic Partitioning, Buddy Systems, Fragmentation, Paging, Segmentation, Address translation. **Placement Strategies:** First Fit, Best Fit, Next Fit and Worst Fit. **Virtual Memory:** Concepts, Swapping, VM with Paging, Page Table Structure, Inverted Page Table, Translation Lookaside Buffer, Page Size, VM with Segmentation with combined paging and segmentation. **Page Replacement Policies:** FIFO, LRU, Optimal, Clock. Swapping issues: Thrashing

Unit 6: I/O and File Management

(4 Hours)

I/O management: I/O Devices - Types, Characteristics of devices, OS design issues for I/O management, I/O Buffering. Disk Scheduling: FCFS, SCAN, C-SCAN, SSTF. List of Tutorials (13): File Management: Concepts, File Organization, File Directories, File Sharing. Record Blocking, Secondary Storage Management, Free Space management, Security.

Syllabus Laboratory

List of Experiments

1. Execution of Basic & Advanced Linux Commands.
2. Write shell script covering – basic arithmetic, control structures, loops, execution of Linux command in shell, command line arguments, functions and arrays. (Title)
3. Write a C /C++ program to solve synchronization problem – Reader writer problem mutex & semaphore.
4. Write a C /C++ program to solve synchronization problem - Producer consumer problem mutex & semaphore.

5. Write a C /C++ program to solve synchronization problem - dining philosopher problem using mutex & semaphore.
6. Implement CPU scheduling algorithms – FCFS , SJF , Round Robin and priority.
7. Implement Banker's algorithm
8. Implement deadlock detection algorithm
9. Implement placement strategies – First fit, Best Fit, Next Fit and Worst Fit.
10. Implement buddy system.
11. . Implement page replacement algorithm – FIFO, LRU and Optimal.
12. Write a C/ C++ program to convert logical/ virtual address into physical address using paging and segmentation.
13. . Implement disk scheduling algorithm – FCFS, SSTF, SCAN, C-SCAN.

Syllabus

Course Project

List of Course Projects

1. Design and implementation of a
 - i. CPU/ Machine Simulation
 - ii. Supervisor Call through interrupt Design multi programming operating system phase 1
2. Design and implementation of a Multiprogramming Operating System: Stage II
 - i. Paging
 - ii. Error Handling
 - iii. Interrupt Generation and Servicing
 - iv. Process Data Structure
3. Design and implementation of a Multiprogramming Operating System: Stage III
 - i. I/O Channels & I/O buffering
 - ii. Multiprogramming
 - iii. I/O Spooling
4. Design multi programming operating system phase 1 with arithmetic & logical instructions
5. Design multi programming operating system phase 3 with swapping.

Course Outcomes

The student will be able to

1. Discuss the functions of a contemporary Operating system with respect to convenience, efficiency and the ability to evolve
2. Implement concurrent abstractions correctly in an OS to solve real world problems.
3. Use various CPU scheduling algorithms to construct solutions to real world problems.

4. Correlate the mechanisms related to deadlock handling in real life situations.
5. Distinguish memory management schemes & file management systems in various ways to improve performance, and analyze the impact of it on system complexity.
6. Design & develop the Operating system from a scratch

CO-PO Mapping

	Program Outcomes (PO)											PSO		
CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2											3	3
CO2	2	2	3	2			2	2					3	3
CO3	2	3	3	2			2	2					3	3
CO4	2	3		2									3	3
CO5	2	3		1									3	3
CO6	2	2	3	2		2	2	2	3	3	3		3	3
Average	2	2.5	3.0	1.8		2.0	2.0	2.0	3.0	3.0	3.0		3.0	3.0

Future Courses Mapping:

Advanced Operating System, Distributed Operating System, Parallel Computing.

Job Mapping:

System Administrator, System Analyst

Books and E-Resources

For Reference Print Book -

1. W. Stallings; 'Operating Systems: Internals and Design Principles'; 9th Edition; Pearson; 2017
2. A. Silberschatz, P. B. Galvin, G. Gagne ; 'Operating System Concepts'; 10th Edition ; Wiley; 2021.

For Reference Electronic Book –

R.H. Arpaci-Dusseau, A. C. Arpaci-Dusseau; 'Operating Systems: Three Easy Pieces'; 1st Edition; CreateSpace Independent Publishing Platform (Arpaci-Dusseau Books, LLC) , 2018. Accessed: June. 9, 2025. [Online]. Available: <https://pages.cs.wisc.edu/~remzi/OSTEP/>

For MOOCs and other learning Resources

R. Ahuja,; 'Hands-on Introduction to Linux Commands and Shell Scripting'; Coursera; <https://www.coursera.org/learn/hands-on-introduction-to-linux-commands-and-shell-scripting/home>; Accessed: June. 9, 2025

CS2308: Software Engineering

Credits: 3

Teaching Scheme Theory: 2 Hours/Week

Lab: 2 Hours/Week

Course Prerequisites: Basic Programming Skills, Object-Oriented Programming Concepts, Problem Solving and Logic Building

Course Objectives:

1. **To introduce foundational concepts and various software development lifecycle models**, including traditional and agile approaches such as Scrum, along with their roles and artifacts.
2. **To develop the ability to elicit, analyze, and document software requirements**, and to understand the structure of Software Requirement Specification (SRS) and use case modeling.
3. **To explain key principles of software design and differentiate between high-level and low-level designs**, focusing on modularity, reusability, and design patterns.
4. **To familiarize students with structural modeling using UML**, including class, object, and use case diagrams, and enable hands-on diagram creation using UML tools.
5. **To enable learners to represent dynamic behavior of software systems using UML behavioral models**, such as sequence, activity, and state diagrams, along with deployment and component diagrams.
6. **To impart practical knowledge of software project management**, including cost estimation, risk management, scheduling, quality assurance, and the use of tools like Trello and JIRA.

Course Relevance:

Software Engineering applies systematic principles and methodologies to design, develop, and maintain reliable, efficient, and scalable software systems, drawing on foundational knowledge from Computer Engineering. It encompasses the full software lifecycle, from requirements gathering and design to implementation, testing, and maintenance, ensuring quality and security within project constraints. The course emphasizes formal techniques, practical tools, and teamwork to build trustworthy software that meets real-world demands.

Syllabus Theory

Unit 1: Introduction to Software Engineering (4 Hours)

Nature and scope of software engineering, Software characteristics and application domains, SDLC models: Waterfall, Spiral, Incremental, Rapid Application Development, Agile methodology overview: Scrum, roles, and artifacts.

Unit 2: Requirements Engineering (4 Hours)

Functional vs Non-functional requirements, Requirement elicitation techniques: interviews, observation, document analysis, Requirement analysis - MoSCoW technique, Writing Software Requirement Specification (SRS), Use case modeling.

Unit 3: Software Design and Design Principles (4 Hours)

High-level vs Software Design, Software design principles: modularity, cohesion, coupling, abstraction, reusability, High-level design, Low-level design, Characteristics of a good design, Design patterns,

Unit 4: Structural Modeling using UML (6 Hours)

Object-Oriented Modeling basics, Class diagram: classes, attributes, methods, associations, multiplicity, Object diagram, Use case diagram, Relationships: generalization, aggregation, composition, Hands-on drawing structural models using UML tools (e.g., Lucidchart & StarUML)

Unit 5: Behavioral Modeling using UML (6 Hours)

Sequence diagram: object interactions and message flow, Activity diagram: workflows and control flow, State diagram: system behavior modeling, Component and Deployment diagrams, Hands-on drawing behavioral models using UML tools (e.g., Lucidchart & StarUML).

Unit 6: Software Project Management (3 Hours)

Project planning and scheduling (Gantt, PERT), Software cost estimation (Function Point, Use Case Point), Risk management and mitigation, Quality assurance processes and metrics Tools: Trello, JIRA (introduction).

**Syllabus
Laboratory**

List of Experiments

List of experiments: (Any SIX)

1. To prepare a Statement Of Work (SOW) document, which addresses the vision, goals and objectives of the real-world problem.
2. To prepare a Software Requirement Specification (SRS) document, based on several types of system requirements, such as functional and non-functional requirements.
3. Create and manage use case diagrams for a real-world application (e.g., Online Shopping System) to capture functional requirements.
4. Design detailed class and object diagrams using UML tools (e.g., StarUML, Visual Paradigm) for a given problem statement such as a Student Management System.
5. Develop sequence diagrams illustrating object interactions and activity diagrams representing workflows for a banking transaction system.

6. Model the dynamic behavior of a system using state diagrams (e.g., ticket booking process in a cinema management system).
7. Prepare a software project plan with scheduling using Gantt charts and PERT charts for a sample project.
8. Perform software cost estimation exercises using Function Point Analysis and Use Case Points for a given software project.
9. Identify potential risks and prepare mitigation strategies for a hypothetical software development project.
10. Hands-on with Project Management Tools: Practice project tracking, task assignment, and progress monitoring using Trello and JIRA on a simulated project.

Syllabus

Course Project

List of Sample Course Projects

Project 1: Comparative Study of Agile vs. Traditional SDLC Models in Real-World Software Development

Analyze project outcomes, productivity, and quality metrics using case studies or surveys.

Project 2: Automated Requirements Elicitation and Analysis Using NLP Techniques

Develop or enhance a tool that applies Natural Language Processing for extracting and validating requirements from stakeholder documents.

Project 3: Design Patterns Effectiveness in Large-Scale Software Systems: A Quantitative Analysis

Study the impact of specific design patterns on code maintainability, performance, and scalability through quantitative metrics.

Project 4: UML-Based Model-Driven Development Framework for Rapid Prototyping

Build and evaluate a framework or tool that automates code generation from UML models to speed up prototyping.

Project 5: Risk Assessment and Mitigation Strategies in Agile Project Management

Develop a risk management framework tailored for agile projects and validate its effectiveness through case studies or simulations.

Project 6: Software Cost Estimation Accuracy Improvement Using Machine Learning Models

Use historical project data to train predictive models for software cost and effort estimation; compare

with traditional techniques.

Course Outcomes

The student will be able to –

- 1) **Analyze and compare different software development lifecycle models**, including agile methodologies like Scrum, to determine their suitability for various projects.
- 2) **Apply requirements elicitation techniques and prepare a detailed SRS**, identifying both functional and non-functional requirements effectively
- 3) **Evaluate software design principles and construct modular, cohesive, and reusable designs**, applying basic design patterns.
- 4) **Construct structural UML diagrams (class, object, use case)** using appropriate tools for visualizing the software architecture.
- 5) **Develop behavioral UML diagrams (sequence, activity, state, component, deployment)** to model system workflows and interactions.
- 6) **Plan and manage software projects using estimation techniques and project management tools**, addressing risks and ensuring quality assurance.

CO-PO Mapping

	Program Outcomes (PO)											PSO		
CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	3	3	3	3	3	3	3	3	3	3	3	3	3
CO2	3	3	3	3	3	3	3	3	3	3	3	3	3	3
CO3	2	2	2	2	2	2	2	2	2	2	2	2	2	2
CO4	1	1	1	1	1	1	1	1	1	1	1	1	1	1
CO5	1	1	1	1	1	1	1	1	1	1	1	1	1	1
CO6	2	2	2	2	2	2	2	2	2	2	2	2	2	2
Average	2.5	2.5	2.66	1.83	2.33	2		2	2.25	2.5	2	2.83	2.83	

CO Attainment levels:

Weights for attainment levels: L1 - Easy - 0.75, L2 - Comfortable - 0.7, L3 – Medium – 0.65, L4 – Somewhat difficult – 0.6, L5 – Difficult – 0.55.

CO1- (L2), CO2 -(L3), CO3 - (L2), CO4 -(L3), CO5 -(L3), CO6 -(L4)

Future Courses Mapping:

The foundational knowledge and skills developed in *Software Engineering* directly support the learning outcomes of the following advanced and interdisciplinary courses:

Software Architecture and Design Patterns, Software Testing and Quality Assurance, Project Management in Software Development, DevOps and Agile Practices, Cloud Computing and Microservices Architecture, Secure Software Development, Human-Computer Interaction, Machine Learning Systems Engineering.

Job Mapping:

Students who complete a course in Software Engineering are well-prepared for careers where planning, designing, developing, and managing complex software systems is essential. Key job roles include:

Software Developer / Software Engineer, Requirements Analyst / Systems Analyst, Software Architect, DevOps Engineer, QA Engineer / Test Automation Engineer, Project Manager /, Scrum Master, Agile Coach, Product Manager (Technical), Software Consultant, Full Stack Developer, Application Support Engineer, R&D Engineer / Academic Researcher

Text Books:

1. **Ian Sommerville**, *Software Engineering*, 10th Edition, Pearson Education.
2. **Roger S. Pressman & Bruce R. Maxim**, *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill.
3. **Tom Pender**, *'UML Bible'*, John Wiley & Sons, 2003, ISBN - 0764526049
4. **Kenneth Rubin**, *'Essential SCRUM: A Practical Guide To The Most Popular Agile Process'*, Addison-Wesley, 2012, ISBN-13: 978-0-13-704329-3.

Reference Books:

1. **SorenLauesen**, *'Software Requirements: Styles and Techniques'*, Addison Wesley, 2002, ISBN 0201745704.
2. **Pankaj Jalote**, *An Integrated Approach to Software Engineering*, 3rd Edition, Springer.
3. **S. L. Pfleeger & Joanne M. Atlee**, *Software Engineering: Theory and Practice*, 4th Edition, Pearson.
4. **Grady Booch, James Rumbaugh, Ivar Jacobson**, *'Unified Modeling Language User's Guide'*, 2nd Edition, AddisonWesley 2005, ISBN – 0321267974.

For MOOCs and other learning Resources

1. NPTEL – Software Engineering- <https://nptel.ac.in/courses/106105087>
2. GeeksforGeeks – Software Engineering- <https://www.geeksforgeeks.org/software-engineering/>
3. Coursera – Software Process and Agile Practices
4. Classroom VolP

CS2309: Mobile Application Development

Credits: 1

Teaching Scheme Lab: 2 Hours/Week

Course Prerequisites:

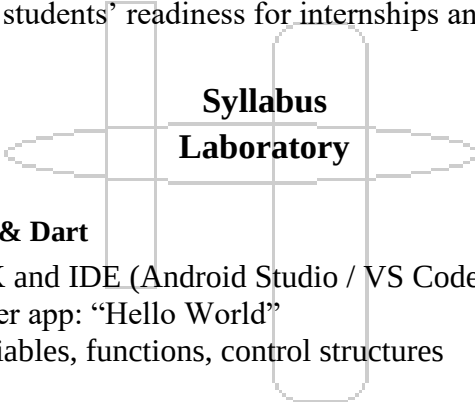
Basic knowledge of programming (any language)

Course Objectives:

1. Understand the basics of Dart and Flutter for cross-platform mobile development
2. Design responsive user interfaces with Flutter widgets
3. Implement navigation, state management, and API integration
4. Build, test, and deploy simple mobile applications

Course Relevance:

The course equips practical skills in cross-platform app development using Flutter, a framework in high demand across industries. It bridges theoretical programming concepts with real-world mobile application design, enhancing students' readiness for internships and projects.



Syllabus Laboratory

List of Experiments

Lab 1: Introduction to Flutter & Dart

- Setting up Flutter SDK and IDE (Android Studio / VS Code)
- Running the first Flutter app: "Hello World"
- Basic Dart syntax: variables, functions, control structures

Lab 2: Flutter Project Structure & Widgets

- Exploring project folders and key files ([pubspec.yaml](#), [main.dart](#))
- Understanding Stateless and Stateful widgets
- Building a basic UI with common widgets ([Text](#), [Container](#), [Row](#), [Column](#))

Lab 3: Layouts and Styling

- Using layout widgets: [Stack](#), [Expanded](#), [SizedBox](#)
- Styling UI elements with themes, colors, and fonts
- Designing a basic login screen

Lab 4: User Input & Forms

- Working with [TextField](#) for user input
- Creating forms using [Form](#) and [TextFormField](#)
- Implementing basic input validation

Lab 5: Navigation and Routing

- Navigating between multiple screens using **Navigator**
- Passing data between screens
- Implementing a simple multi-page app

Lab 6: State Management Basics

- Introduction to state management: `setState`, Stateful widgets
- Handling user interactions and updating the UI
- Building a counter app with dynamic state

Lab 7: Lists and Dynamic Content

- Displaying dynamic lists using **ListView** and **ListView.builder**
- Creating simple item cards with **Card** widget

Lab 8: Fetching Data from APIs

- Using the **http** package to make network requests
- Fetching and displaying data from a REST API
- Basic JSON parsing in Flutter

Lab 9: Local Data Storage

- Introduction to local storage: **shared_preferences**
- Storing and retrieving simple data (e.g., login status, preferences)
- Basics of offline support

Lab 10: Final App Build and Deployment

- Consolidating concepts to build a mini project/app
- Building the APK for deployment
- Debugging and best practices recap

Course Outcomes

1. Understand the architecture and basic components of Flutter and Dart to build cross-platform apps.
2. Design and implement responsive UIs using Flutter widgets and layout structures.
3. Apply form validation and user interaction handling techniques to develop functional mobile apps.
4. Implement navigation and routing to create multi-screen apps with data transfer.
5. Integrate REST APIs and local storage (shared preferences) for data handling in apps.
6. Develop and deploy a basic Flutter-based mobile application meeting user requirements.

CO-PO Mapping

	Program Outcomes (PO)												PSO			
CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3	PSO 4
CO1	2	2	3									2	3		2	
CO2	2	3	2									2	3		2	
CO3	3	3	3				2	2				2	3	2	2	2
CO4	3	3	3	3			2	2				2	3	2	2	2
CO5	3	3	2									2	3		2	
CO6	2	3	3									2	3		2	
Average	2.5	3.0	2.66	3.0			2.0	2.0				2.0	3.0	2.0	2.0	2.0

Future Courses Mapping:

Mobile Computing, Web Technologies

Job Mapping:

Mobile Application Developer, Front-end Developer (Mobile-focused), UI/UX Developer for mobile apps, Freelancer / Independent App Developer, Mobile Solutions Engineer

Books and E-Resources

For Reference Print Book -

1. Alessandro Biessek, "Flutter for Beginners, Packt Publishing, 2020 (Second Edition), ISBN: 978-1800565997
2. Frank Zammetti, "Practical Flutter: Improve Your Mobile Development with Google's Latest Open-Source SDK", Apress, 2019, ISBN: 978-1484249717

For MOOCs and other learning Resources

1. [SWAYAM Plus](#)
2. [The Complete Flutter Development Bootcamp with Dart | Udemy](#)
3. [Flutter and Dart: Developing iOS, Android, and Mobile Apps | Coursera](#)

Multidisciplinary Minor**

CSM: Microcontroller

Credits: 3

Teaching Scheme Theory: 2 Hours/Week

Tutorial: 1 Hour/Week

Course Prerequisites: Computer organization & Architecture, Logic Design, Microprocessor,

Course Objectives:

1. To understand the architecture and features of PIC microcontrollers.
2. To learn memory organization and I/O programming using TRIS and LAT.
3. To implement timers, interrupts, and serial communication in PIC.
4. To study the architecture and register a set of ARM Cortex-M processors.
5. To understand data types, and addressing modes and instructions set in ARM.
6. To design real-time embedded applications using sensors and communication protocols.

Course Relevance: This is a basic course for Computer Science and Engineering. The course plays an essential role while modeling problems in computer science and engineering. Microcontroller course is highly relevant for Computer Engineering especially in today's world of smart devices, automation, and embedded intelligence.

**Syllabus
Theory**

Section 1: Topics/Contents

Unit 1: Introduction to PIC Microcontrollers (4 Hours)

Evolution and features of PIC microcontrollers Overview of PIC18 family, Architecture and pin configuration, Memory organization: RAM, ROM, EEPROM, Configuration registers and status flags

Unit 2: Instruction Set and I/O Programming (5 Hours)

Arithmetic, logic, and branch instructions, Stack operations and program counter, I/O port programming using TRIS and LAT registers, Bit manipulation and time delay generation, Assembly and embedded C programming basics.

Unit 3: Timers, Interrupts, and Serial Communication (5 Hours)

Timer modules (Timer0, Timer1, Timer2), Interrupt types and handling, UART programming and RS232 interfacing, Basics of SPI and I2C communication

Section 2: Topics/Contents

Unit 4: ARM Microcontroller

(5 Hours)

Overview of ARM processor families (Cortex-M); RISC vs CISC; Block and Functional Diagram of ARM Controller, Pin Configuration of ARM Microcontroller. ARM pipeline and instruction set basics.

Unit 5: ARM Data Organization

(5 Hours)

Instruction Set of ARM Microcontroller, Addressing Modes, Data Types, Register Set, Applications of ARM Microcontroller.

Unit 6: Advanced Application

(4 Hours)

Mini real-time embedded systems: digital thermometer, sensor node, home automation controller. ARM in IoT: connectivity and cloud integration

**Syllabus
Tutorials**

List of Tutorials

1. PIC microcontroller architecture and pin configuration
2. Memory organization: RAM, ROM, EEPROM, configuration registers
3. PIC instruction set
4. Stack operations and program counter behavior in PIC
5. I/O programming using TRIS and LAT registers
6. Bit manipulation and software delay generation
7. Timer configuration in PIC
8. Interrupt handling and flag management in PIC
9. UART programming and RS232 serial communication
10. SPI and I2C communication protocols
11. ARM Cortex-M microcontroller architecture and pipeline
12. ARM instruction Set
13. ARM Addressing modes

14. Real-time embedded applications

Course Outcomes

After successful completion of this course, students will be able to:

1. Describe the evolution, architecture, features, and memory organization of PIC microcontrollers.
2. Elaborate assembly and embedded C programs for arithmetic, logic, stack, and I/O operations using TRIS and LAT registers in PIC.
3. Describe timers, interrupts, UART, SPI, and I²C for real-time applications in PIC microcontrollers.
4. Discuss the architecture, processor families, and pipeline features of ARM Cortex-M microcontrollers.
5. Interpret the instruction set, addressing modes, data types, and register usage in ARM Cortex-M microcontrollers for embedded system development.
6. Discuss real world application of ARM

CO-PO Mapping

CO/PO	Program Outcomes (PO)												PSO			
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3	PSO 4
CO1	3	1		1								1	1		3	1
CO2	3	2				1			2		2	1	2		3	2
CO3	3	2	2									1	2		3	2
CO4	3	2	2	3								1	2		3	2
CO5	3	2	1						1		2	1	2	1	3	2
CO6	3	2	1	1		1	1	1			2	1	2	3	3	2
Average	3	1.8	1	0.8	0	0.3	0.1	0.1	0.5	0	1.3	1	1.8	0.7	3	1.8

Future Courses Mapping: Parallel Computing, Operating System, IoT, Embedded System

Job Mapping:

Embedded Systems Engineer, Firmware Developer, IoT Developer, Automation Engineer, Robotics Engineer

Textbook

1. "PIC Microcontroller and Embedded Systems: Using Assembly and C for PIC18", 1st Edition, Muhammad Ali Mazidi, Rolin D. McKinlay & Danny Causey (Pearson/Prentice Hall, 2008)
2. "Design with PIC Microcontrollers", 1st Edition, John B. Peatman (Prentice Hall, 1997–98)
3. "Embedded C Programming and the Microchip PIC", 1st Edition, Richard H. Barnett, Sarah Cox & Larry O'Cull (Delmar Cengage Learning, November 2003)
4. "PIC Microcontroller Project Book", 2nd Edition, John Iovine (McGraw-Hill, March 29 2004)
5. Intel 8-bit Microcontroller manual.

6. Ajay Deshmukh, "Microcontrollers — (Theory and application)" , 2004, TMH,ISBN 0-07-058595-4
7. J. Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, Newnes

